

Comprehensive Study Guide

CME 295: Transformers & LLMs — Lecture 8

LLM Evaluation: Human Ratings, Rule-Based Metrics, LLM-as-a-Judge, Agentic Failures, Benchmarks

Stanford CME 295 by Afshine & Shervine Amidi · For: Applied AI/ML Engineers

Contents

1	Topic Map	2
2	Deep-Dive Notes	4
2.1	What Does “Evaluation” Mean?	4
2.2	Human Ratings and Inter-Rater Agreement	4
2.2.1	Inter-Rater Agreement and Cohen’s Kappa	4
2.3	Rule-Based Metrics	5
2.3.1	METEOR (Banerjee & Lavie, 2005)	5
2.3.2	BLEU and ROUGE	5
2.3.3	Limitations of Rule-Based Metrics	6
2.4	LLM-as-a-Judge (LaaJ)	6
2.4.1	Two Variants	7
2.4.2	Three Biases	7
2.4.3	Best Practices Summary	7
2.4.4	Revised Evaluation Workflow	8
2.5	Factuality Evaluation	8
2.6	Agentic Failure Mode Taxonomy	8
2.6.1	Step 1: Tool Prediction Failures	8
2.6.2	Step 2: Tool Call Failures	9
2.6.3	Step 3: Response Generation Failures	9
2.6.4	Common Issue Categories	10
2.7	Standard Benchmarks	10
2.7.1	MMLU: Knowledge Benchmark	10
2.7.2	AIME: Math Reasoning Benchmark	11
2.7.3	PIQA: Common-Sense Reasoning	11
2.7.4	SWE-bench: Coding Benchmark	11
2.7.5	HarmBench: Safety Benchmark	11
2.7.6	tau-bench: Agentic Benchmark	11
2.8	Benchmark Meta-Topics	12
3	Related Concepts You Should Also Know	13
4	Build Projects	14
5	Explain It Back	15
6	Interview Practice Questions & Answers	16
7	Self-Assessment Test	18

Topic Map

1. Evaluation Scope and Motivation

- 1.1 Output quality vs. system performance
- 1.2 Why free-form text is hard to evaluate

2. Human Ratings and Inter-Rater Agreement

- 2.1 Why human ratings are the gold standard
- 2.2 Subjectivity and inter-rater agreement
- 2.3 Agreement rate by chance; why raw agreement is misleading
- 2.4 Cohen's kappa derivation; Fleiss's kappa, Krippendorff's alpha
- 2.5 Cost and speed limitations

3. Rule-Based Metrics

- 3.1 Reference-based evaluation setup
- 3.2 METEOR: formula, F-score component, ordering penalty
- 3.3 BLEU: precision-focused, brevity penalty
- 3.4 ROUGE: recall-focused, summarization
- 3.5 Limitations: stylistic variation, low correlation, still requires labels

4. LLM-as-a-Judge (LaaJ)

- 4.1 Concept, inputs (prompt, response, criteria), outputs (rationale + score)
- 4.2 Structured output enforcement (guided decoding)
- 4.3 Benefits: no reference needed, interpretability
- 4.4 Pointwise vs. pairwise variants
- 4.5 Three biases: position, verbosity, self-enhancement
- 4.6 Best practices: crisp guidelines, binary scale, rationale first, low temperature, calibration
- 4.7 Revised workflow: LLM → LaaJ → human spot-checks

5. Factuality Evaluation

- 5.1 Two-step pipeline: fact decomposition + fact checking
- 5.2 Importance weights and weighted score formula

6. Agentic Failure Mode Taxonomy

- 6.1 Step 1 (Tool prediction) failures: doesn't use tool, hallucinates tool, wrong tool, wrong argument
- 6.2 Step 2 (Tool call) failures: wrong/error response, no response
- 6.3 Step 3 (Response generation) failures: wrong final response
- 6.4 Causes and remedies for each

7. Benchmarks

7.1 Knowledge: MMLU (57 tasks, 4-choice, hardcoded match)

7.2 Reasoning — math: AIME (3-digit answer, hardcoded match)

7.3 Reasoning — common sense: PIQA (2-choice, physical world)

7.4 Coding: SWE-bench (GitHub issues, test pass rate)

7.5 Safety: HarmBench (attack success rate, classifier)

7.6 Agentic: tau-bench (two domains, pass-hat-k metric)

8. Benchmark Meta-Topics

8.1 Pareto frontier (quality vs. cost/latency/safety)

8.2 Data contamination and precautions (hash, blocklist, new versions)

8.3 Goodhart’s Law and why benchmarks are not the goal

Deep-Dive Notes

What Does “Evaluation” Mean?

Two Axes of Evaluation (This Lecture: Output Quality)

- **Output quality:** How good is the response itself? Instruction following, coherence, factuality, relevance, tone, safety. **Focus of this lecture.**
- **System performance:** Latency, pricing, uptime/reliability, throughput. Important for production but outside the scope of this lecture.

Why is LLM evaluation hard? LLMs produce free-form text — code, math, natural language, creative writing, structured data. There is no single universal metric. The “correct” response is often not unique (many valid ways to express the same idea), and quality is partially subjective.

Human Ratings and Inter-Rater Agreement

Human Rating: The Gold Standard

Ideal evaluation: for each LLM output, have a human rate it according to a specified criterion. This is the closest approximation of ground truth because LLMs are ultimately deployed for humans.

Limitations:

1. **Subjectivity:** Different raters may reasonably disagree on the same response.
2. **Speed:** Rating 1,000 outputs takes significant human time.
3. **Cost:** Human annotators are expensive, especially domain experts.

Inter-Rater Agreement and Cohen’s Kappa

Why Raw Agreement Rate Is Misleading

Suppose two raters A and B each rate responses independently. A gives a positive rating with probability p_A and B with probability p_B .

The probability they agree by *pure chance*:

$$P(\text{agree by chance}) = p_A p_B + (1 - p_A)(1 - p_B)$$

If $p_A = p_B = 0.5$: chance agreement = $0.25 + 0.25 = 0.5 = 50\%$.

A 50% agreement rate is completely uninformative — it’s what you’d expect from random guessing. Raw agreement rate doesn’t account for baseline chance agreement, which depends on how often raters tend to say “yes” vs. “no.”

Cohen’s Kappa

$$\kappa = \frac{P_{\text{observed}} - P_{\text{expected}}}{1 - P_{\text{expected}}}$$

where P_{observed} = fraction of times raters agree, and P_{expected} = expected agreement by chance given each rater’s marginal distribution.

Interpretation:

- $\kappa = 1$: perfect agreement.
- $\kappa = 0$: agreement no better than chance.

- $\kappa < 0$: agreement worse than chance (raters are systematically disagreeing).

Variants:

- **Cohen’s kappa**: 2 raters.
- **Fleiss’s kappa**: ≥ 2 raters.
- **Krippendorff’s alpha**: handles missing data, ordinal scales, multiple raters.

Practical Use of Kappa

Track kappa during annotation projects as a “health metric.” If kappa is unsatisfactory (e.g., < 0.6), hold alignment sessions to clarify guidelines. Kappa does not tell you if the ratings are *correct*, only if raters are *consistent*.

Rule-Based Metrics**Reference-Based Evaluation Setup**

Instead of rating every LLM output, write reference (ideal) outputs *once* for a fixed set of prompts. Then compare LLM outputs to these references using automated metrics. The reference is fixed; the LLM can be iterated on.

Key benefit: Enables rapid comparison of model versions without repeated human annotation.

METEOR (Banerjee & Lavie, 2005)**METEOR: Metric for Evaluation of Translation with Explicit ORdering**

$$\text{METEOR} = F_{\alpha}(P, R) \times (1 - \text{Penalty})$$

where:

- $F_{\alpha} = \frac{PR}{\alpha P + (1-\alpha)R}$ is a weighted harmonic mean (F-score) of precision and recall over matched unigrams.
- P = precision: fraction of predicted unigrams matching the reference.
- R = recall: fraction of reference unigrams matched in the prediction.
- $\text{Penalty} = \gamma \cdot (C/M)^{\beta}$ where C = number of contiguous matched chunks, M = total matched unigrams, γ and β are hyperparameters.

The penalty increases when matched unigrams appear in many separate chunks (scattered order), incentivizing the same word ordering as the reference.

METEOR’s advantage over BLEU: Accounts for synonyms and morphological variants (“running” matches “run”), not just exact word matches.

BLEU and ROUGE**BLEU (Papineni et al., 2002)**

Precision-focused n-gram overlap between prediction and reference, plus a brevity penalty (BP) to prevent gaming the metric with very short predictions:

$$\text{BLEU} = \text{BP} \times \exp\left(\sum_{n=1}^N w_n \log p_n\right)$$

where p_n is the precision of n -gram matches and w_n are weights (typically equal). Higher BLEU = better.

Primarily used for machine translation. Not suitable for tasks where many valid outputs exist.

ROUGE (Lin, 2004)

Recall-focused n-gram overlap. ROUGE-N measures n-gram recall; ROUGE-L measures longest common subsequence. Primarily used for summarization. Higher ROUGE = better.

Limitations of Rule-Based Metrics

Why Rule-Based Metrics Are Insufficient for Modern LLMs

1. **No stylistic variation:** These three sentences convey the same meaning but have zero n-gram overlap: (a) “A plush teddy bear can comfort a child during bedtime.” (b) “Soft stuffed bears often help kids feel safe as they fall asleep.” (c) “Many youngsters rest more easily at night when they cuddle a gentle toy companion.” BLEU/ROUGE would score (b) and (c) as 0 against reference (a).
2. **Low correlation with human ratings:** Despite years of hyperparameter tuning, these metrics correlate poorly with human judgment on modern, high-quality LLM outputs.
3. **Still requires human references:** You need human-written ideal outputs to compute these metrics — the cost is deferred, not eliminated.

LLM-as-a-Judge (LaaJ)

LLM-as-a-Judge (Zheng et al., 2023)

Use a capable LLM as an automated evaluator. The judge receives:

- The **prompt** used to generate the response.
- The **model response** to be evaluated.
- The **evaluation criteria** (e.g., relevance, factuality, safety).

And outputs:

- A **rationale** (1–2 sentences explaining the judgment).
- A **score** (e.g., binary PASS/FAIL, or 1–5).

Key insight: LLMs trained on human data already contain implicit human preferences. They can approximate human judgment without requiring explicit human labels for each evaluation.

Why Output Rationale *Before* Score

Asking the judge to write its rationale before the score mirrors chain-of-thought reasoning (Lecture 6). The rationale forces the judge to “think” about the response before committing to a score, empirically improving judgment quality and reproducibility.

Structured Output Enforcement

LLM outputs are stochastic — there’s no guarantee the judge produces parseable JSON or a valid score. Use **structured output** / **guided decoding** (constrained decoding from Lecture 3) to enforce the output format.

In practice: define the expected output schema (e.g., a Pydantic class with fields `rationale: str` and `score: Literal[0, 1]`), then pass it via the provider’s API (`text_format=Response` in OpenAI, or equivalent in Anthropic/Google).

Two Variants

Pointwise LaaJ

Evaluate a single response: “Is this response good?” Returns a quality score. Use when you need to evaluate a model’s absolute performance, not relative.

Pairwise LaaJ

Compare two responses: “Is response A or B better?” Returns a preference. Use for A/B testing models, generating synthetic preference data for RLHF reward model training (Lecture 5).

Three Biases

Position Bias

Symptom: In pairwise evaluation, the judge systematically favors whichever response appears first.

Remedy: Run the same comparison twice with the responses in reversed order. If the judge changes its answer depending on position, the result is unreliable. Average or take majority vote across both orderings.

Verbosity Bias

Symptom: The judge prefers longer, more verbose responses even when a shorter, more concise response is objectively better.

Remedies:

- Explicit guidelines: “Do not prefer a response simply because it is longer.”
- Few-shot examples showing short correct answers beating verbose incorrect ones.
- Post-processing: penalize score by response length.

Self-Enhancement Bias

Symptom: When using the same model as both generator and judge, the model prefers its own outputs over equally good or better alternatives.

Remedy: Use a different model as judge. Ideally a larger, more capable model. In practice, all frontier models share similar training data, so this bias is reduced but not eliminated.

Rule of thumb: Judge model $\neq$ generator model. Judge model should generally be stronger.

Best Practices Summary

LLM-as-a-Judge Best Practices

1. **Crisp guidelines:** Be explicit about what counts as good/bad for each criterion. Vague guidelines produce noisy scores.
2. **Binary scale:** Pass/fail is easier to calibrate than 1–5 scales. Fewer choices = less noise.
3. **Rationale before score:** Forces chain-of-thought; improves quality.
4. **Mitigate known biases:** Position (swap and average), verbosity (explicit guidelines), self-enhancement (different judge model).
5. **Calibrate with human ratings:** Periodically collect human ratings on a sample, compute correlation with judge scores, and adjust the judge prompt if correlation is low.
6. **Low temperature for reproducibility:** Use $T \approx 0.1$ – 0.2 to reduce variance across

evaluation runs.

Revised Evaluation Workflow

Practical Workflow

1. **Fast path:** LLM-as-a-Judge scores all outputs automatically. Use this for frequent, high-volume evaluation during development.
2. **Calibration:** Periodically collect human ratings on a random sample (~50–200 examples). Run correlation analysis. If LaaJ diverges from humans, revise the judge prompt.
3. **Key insight:** Don't over-optimize for LaaJ scores — the judge is an approximation of human judgment, not the ground truth. Reward hacking applies here too.

Factuality Evaluation

Two-Step Factuality Scoring (Wei et al., 2024)

Step 1 — Fact decomposition: Use an LLM to convert the free-form text output into a list of atomic, checkable claims.

Example: “Teddy bears, first created in the 1920s, were named after President Theodore Roosevelt...” →

1. Teddy bears were first created in the 1920s.
2. Teddy bears were named after President Theodore Roosevelt.
3. Theodore Roosevelt was on a hunting trip where a bear was captured.
4. Theodore Roosevelt proudly wanted to shoot the captured bear.

Step 2 — Fact checking: For each atomic claim, verify it against a trusted source (RAG over Wikipedia, web search, or a specialized fact-checking model). Each claim is binary: correct (1) or incorrect (0).

Weighted aggregation:

$$\text{Factuality score} = \sum_{i=1}^n \alpha_i \cdot c_i$$

where α_i = importance weight of fact i (can be uniform), $c_i \in \{0, 1\}$ = correctness of fact i , and $\sum_i \alpha_i = 1$.

Example: Facts 2 and 3 are correct (weights 0.4 and 0.2); Facts 1 and 4 are wrong (weights 0.3 and 0.1). Score = $0.4 + 0.2 = 0.6$.

Why Weighted Factuality?

A single factual error in a long text does not make the entire text wrong. By decomposing into atomic facts and weighting by importance, we get a nuanced score that reflects both the number and significance of errors.

Agentic Failure Mode Taxonomy

Agentic systems fail at three layers of the tool-call pipeline:

Step 1: Tool Prediction Failures

Table 1: Tool Prediction Failures

Symptom	Potential Causes	Remedies
Doesn't use tool; issues a direct "punt"	Tool router error (recall failure); model doesn't know how to use tool	Retrain tool router; SFT-train model or improve prompt/API description
Hallucinates a tool (calls non-existent function)	Model too weak; API naming is illogical; instructions unclear	Upgrade model; rename/revamp API; iterate on top-level instructions
Uses the wrong available tool	Tool router confusion; model can't distinguish between similar tools	Retrain tool router; clarify scope boundaries in each tool's docstring
Right tool, wrong arguments	Argument can't be inferred from context; model doesn't understand argument format	Add a helper tool to supply missing context; SFT-train on correct examples; improve argument documentation

Table 2: Tool Call (Backend) Failures

Symptom	Cause	Remedy
Wrong or error response (e.g., ValueError)	Bug in tool implementation	Fix the tool code
No response (tool returns None/void)	Missing return statement or silent failure	Always emit a structured response, even empty JSON {} conveys "nothing found"

Step 2: Tool Call Failures

Always Return Something from Tools

An empty JSON (`{}`) is meaningful: "I ran but found nothing." A return value of `None` is not: "Something unknown happened." The LLM grounding on `None` may hallucinate a positive confirmation, silently failing the user.

Step 3: Response Generation Failures

Table 3: Response Generation Failures

Symptom	Potential Causes	Remedies
Final response contradicts or ignores tool output	Model lacks grounding; tool response floods context; tool output format is uninterpretable	Upgrade generator LLM; trim backend response; use structured, labeled output format (e.g., dataclass)

Common Issue Categories

Debugging Agents: Two Root Cause Buckets

Modeling issues:

- Weak reasoning or grounding capabilities.
- Context window overwhelmed with irrelevant or poorly-formatted information.
- Tool selection/usage not well-learned (SFT data or prompts needed).

Tool issues:

- Implementation bug (wrong output values).
- Output format not interpretable by the LLM.

Approach: Be methodical. Categorize errors in a spreadsheet. Triage by frequency. Fix the most common category first. Use reasoning chain logs for root cause analysis.

Standard Benchmarks

Table 4: Overview of Standard LLM Benchmarks

Category	Benchmark	Description	Evaluation Criteria	Method
Knowledge	MMLU	57 tasks, 4-choice MCQ, spanning science/law/medicine	Correct letter (A/B/C/D)	Hardcoded match
Math reasoning	AIME	30 olympiad-level math problems	Correct 3-digit integer	Hardcoded match
Common-sense reasoning	PIQA	20K everyday physics situations, 2-choice	Correct Sol1/Sol2	Hardcoded match
Coding	SWE-bench	2,294 real GitHub issues across 12 Python repos	All tests pass after LLM patch	Hardcoded match
Safety	HarmBench	510 harmful behaviors (standard/copyright/contextual/multimodal)	Attack Success Rate (ASR)	Classifier
Agentic	tau-bench	2 domains (air-line/retail), real APIs + policies, LLM-simulated users	Reward + pass-hat-k	Hardcoded match

MMLU: Knowledge Benchmark

MMLU (Hendrycks et al., 2020)

Massive Multitask Language Understanding. 57 diverse tasks including elementary math, US history, CS, law, medicine. Each question has exactly 4 answer choices (A/B/C/D). Evaluation: correct letter. No LLM-as-a-Judge or ambiguity. Primarily reflects pre-training quality — does the model know the relevant facts?

AIME: Math Reasoning Benchmark

AIME

American Invitational Mathematics Examination. Used to qualify for the US Math Olympiad. Problems require multi-step mathematical reasoning. Answer is always a 3-digit integer — trivially parseable and verifiable. Ideal for testing reasoning models (Lecture 6). New problems published each year, reducing data contamination risk.

PIQA: Common-Sense Reasoning

PIQA (Bisk et al., 2019)

Physical Interaction: Question Answering. 20,000 everyday physical reasoning questions with 2 choices (Sol1/Sol2). Tests whether models understand how the physical world works (e.g., how to vacuum up a small object with a hairnet vs. a solid seal). Hardcoded match against ground truth.

SWE-bench: Coding Benchmark

SWE-bench (Jimenez et al., 2023)

2,294 real GitHub issues from 12 popular Python repositories. Each issue has a merged PR with associated tests. The LLM's task: generate a code patch. Evaluation: do the tests pass after applying the patch? Test-driven development applied to LLM evaluation. SWE-bench Verified is a cleaned subset where problem statements are human-validated.

HarmBench: Safety Benchmark

HarmBench (Mazeika et al., 2024)

510 unique harmful behaviors across four categories:

- **Standard:** Conventional harmful content (e.g., how to pick a lock).
- **Copyright:** Generate copyrighted material.
- **Contextual:** Context-dependent harmful behaviors (text modality).
- **Multimodal:** Harmful behaviors requiring image input.

Metric: **Attack Success Rate (ASR)** — fraction of prompts that elicit the harmful behavior from the model. Uses a trained classifier to detect successful attacks. The only major benchmark here that isn't hardcoded-match.

Important note: Safety benchmarks are less universally comparable across providers because each has its own safety policy. Interpret results relative to the provider's stated policy.

tau-bench: Agentic Benchmark

tau-bench / tau-squared-bench (Yao et al., 2024)

Tool-Agent-User Interaction benchmark. Tests end-to-end agentic task completion in two domains:

- **Airline agent:** 500 users, 300 flights, 2000 reservations, ~10 tools, 50 tasks.
- **Retail agent:** 500 users, 50 products, 1000 orders, ~10 tools, 115 tasks.

User interaction is **LLM-simulated** (a separate powerful model plays the user) because

multi-turn conversations cannot be hardcoded. Reward is computed against the database state after the interaction.

pass-hat-k: The Agent Reliability Metric

Contrast with pass@k (at least 1 of k succeeds):

$$\widehat{\text{pass@}k} = P(\text{all } k \text{ attempts succeed}) = \left(\frac{c}{n}\right)^k$$

For agents deployed in high-stakes domains (e.g., airline reservation changes), you need *consistent* success, not just occasional success. An agent that succeeds 70% of the time would give customers different outcomes on different attempts — unacceptable. pass-hat-k penalizes inconsistency.

Benchmark Meta-Topics

Pareto Frontier

The set of benchmark configurations that are Pareto-optimal with respect to multiple objectives. Common trade-offs:

- Quality vs. cost/latency (most common in model selection)
- Quality vs. safety
- Quality vs. context length

Models on the Pareto frontier are non-dominated — no other model is better on all objectives simultaneously.

Data Contamination

Problem: If benchmark questions appear in the training data, the model memorizes answers rather than demonstrating genuine capability.

Precautions:

- Use **hash identifiers** for benchmark examples to detect if they were seen during training.
- For tool-use benchmarks: use a **blocklist** to prevent the model from accessing websites containing benchmark solutions.
- Evaluate on **newer test versions** that post-date training (e.g., AIME 2025 problems cannot be in a model trained on data up to 2024).

Goodhart's Law

“When a measure becomes a target, it ceases to be a good measure.” — Goodhart's Law

Applied to LLM benchmarks: once a benchmark becomes widely used as the *goal* of training, models overfit to it without gaining the underlying capability. Example: training on math benchmark styles improves AIME scores without improving general mathematical reasoning.

Practical implication: Use benchmarks as one signal among many. Complement with Chatbot Arena (human head-to-head), your own domain-specific evaluations, and actual user feedback.

Related Concepts You Should Also Know

1. **MT-Bench:** A multi-turn benchmark for chat assistants. Questions are designed to require multi-turn reasoning and instruction following. Used in the original LLM-as-a-Judge paper (Zheng et al., 2023). Evaluates aspects like math, writing, roleplay, coding, and reasoning across 8 categories.
2. **G-Eval / GPT-4 Evaluator:** A family of LaaJ approaches that use GPT-4 (or equivalent) as the judge with a chain-of-thought prompt. Used for evaluating summarization, dialogue, and other tasks where reference-based metrics fail. Showed higher correlation with human ratings than ROUGE/BLEU.
3. **Reward Model as Judge:** In preference tuning (Lecture 5), the reward model is effectively a specialized form of LaaJ trained on human preference data. The difference: a general LaaJ uses a general-purpose LLM without task-specific fine-tuning; a reward model is fine-tuned specifically for the target evaluation dimension.
4. **LMSYS Chatbot Arena / LMArena:** A live A/B testing platform where users submit real queries and vote on model quality. The ELO-based ranking provides human preference signal at scale. Discussed in Lectures 4 and this lecture as an organic complement to automated benchmarks.
5. **HELM (Holistic Evaluation of Language Models):** Stanford’s benchmark suite covering 16+ scenarios with 7+ metrics each. Emphasizes multi-dimensional evaluation rather than single-score comparisons.
6. **BIG-bench:** Google’s large-scale benchmark with 200+ tasks. Uses hash-based contamination detection. Covers reasoning, world knowledge, language, social reasoning, and more.
7. **FActScore (Min et al., 2023):** An earlier version of the fact decomposition + fact checking approach described in class. Decomposes model-generated biographies into atomic claims and verifies each against Wikipedia. The two-step factuality pipeline in the lecture builds directly on this approach.
8. **Calibration Error:** The discrepancy between a model’s stated confidence and its actual accuracy. A well-calibrated model that says it’s 80% confident should be right 80% of the time. LLMs are often poorly calibrated. Evaluating calibration is separate from evaluating output quality but important for high-stakes applications.
9. **Red Teaming:** Proactively stress-testing a model by deliberately trying to elicit harmful, incorrect, or undesirable outputs. HarmBench and similar safety benchmarks automate a subset of red teaming. Human red teaming (using creative, context-specific attacks) complements automated benchmarks.
10. **Evals-as-Code:** The practice of writing evaluations as reproducible code (Python scripts or CI pipelines) that run automatically on model updates. Enables rapid regression detection. Companies like Anthropic and OpenAI maintain large eval suites internally.

Build Projects

1. **LLM-as-a-Judge Pipeline** (*4–6 hours, very high signal for KnowledgeOS*)
Build a reusable LaaJ evaluation pipeline for your KnowledgeOS RAG system (or AgentPitch). Define 3–4 evaluation dimensions (e.g., factuality, relevance, completeness, tone). For each, write a crisp LaaJ prompt with explicit binary pass/fail criteria. Use structured output (`Literal[0, 1]`) to enforce parseable scores. Implement position-swapping for pairwise evaluations. Collect 50 human ratings on the same queries and compute Spearman correlation between LaaJ scores and human ratings. Report the correlation per dimension and iterate on the worst-performing judge prompt.
2. **Factuality Scorer** (*3–5 hours, high signal*)
Implement the two-step factuality pipeline on a corpus of LLM-generated text about a domain you know well. Step 1: decompose outputs into atomic facts using an LLM. Step 2: verify each fact against Wikipedia (using a RAG retriever) and return binary correct/incorrect. Implement uniform and importance-weighted scoring. Validate by manually reviewing a sample of 20 outputs and comparing your automated scores to manual judgments. Report your kappa with yourself (check consistency by scoring the same 10 examples twice with a 1-week gap).
3. **Agentic Failure Mode Audit** (*3–4 hours, directly applicable*)
For your AgentPitch system (or any agent you’ve built), create a structured test suite covering all 7 failure modes from the lecture. For each failure mode: (a) write 3 test prompts that should trigger that failure, (b) run the agent, (c) categorize the actual failure type, (d) identify the root cause, (e) implement the recommended remedy. Measure pass rate before and after each remedy. This is the exact debugging methodology used in production agentic systems.

Explain It Back

1. A startup asks you to evaluate their new LLM. They say they'll just use BLEU score. What do you tell them? Cover: why BLEU fails for modern LLMs, what you'd use instead (LaaJ), and how you'd set up the evaluation pipeline end-to-end.
2. Explain why raw agreement rate between two raters is misleading. Derive the formula for agreement by chance when rater A says yes with probability p_A and rater B says yes with probability p_B . Then explain intuitively what Cohen's kappa measures.
3. Walk through all 7 agentic failure modes using a concrete example of your own (not the teddy bear). For each, state the symptom, the most likely cause, and the remedy you'd apply first.
4. You are comparing two LLMs. LLM-A scores 87% on MMLU; LLM-B scores 79%. But users on Chatbot Arena prefer LLM-B 60% of the time. How do you explain this? What does Goodhart's Law have to do with it, and what additional evaluation would you run?

Interview Practice Questions & Answers

L1 — What is LLM-as-a-Judge and what are its two main advantages over rule-based metrics?

Answer: LLM-as-a-Judge uses a capable LLM to evaluate another LLM's outputs. The evaluator receives the prompt, the response, and evaluation criteria, then outputs a rationale and a score. Two main advantages over BLEU/ROUGE/METEOR: (1) *No reference needed* — the judge uses its pre-trained knowledge of human preferences without requiring hand-written ideal answers; (2) *Interpretability* — unlike opaque numeric scores from formula-based metrics, the judge explains why it gave a particular score via the rationale, enabling actionable debugging.

L1 — Name and explain the three biases of LLM-as-a-Judge.

Answer: (1) *Position bias*: in pairwise evaluation, the judge systematically prefers whichever response appears first in the prompt. Remedy: run the comparison both ways and average/take majority. (2) *Verbosity bias*: the judge favors longer responses regardless of quality. Remedy: explicit anti-verbosity guidelines, few-shot examples where short beats long, length penalty. (3) *Self-enhancement bias*: when the judge and generator are the same model, the judge prefers its own generated outputs even against objectively better alternatives. Remedy: use a different model as judge (preferably larger and more capable).

L2 — Why is raw inter-rater agreement rate misleading? What does Cohen's kappa measure?

Answer: Raw agreement conflates genuine agreement with chance agreement. Two raters who each randomly say “yes” 50% of the time will agree 50% of the time purely by chance ($0.5 \times 0.5 + 0.5 \times 0.5 = 0.5$). If both raters are biased toward “yes” (say, $p_A = p_B = 0.9$), chance agreement = $0.9^2 + 0.1^2 = 0.82$ — so a raw agreement of 85% would be almost meaningless. Cohen's kappa corrects for this: $\kappa = (P_{\text{obs}} - P_{\text{chance}}) / (1 - P_{\text{chance}})$. It measures how much better actual agreement is than chance agreement, normalized to $[0, 1]$ for positive agreement above chance. $\kappa = 0$ means no improvement over random; $\kappa = 1$ means perfect agreement.

L2 — Explain the factuality scoring pipeline. Why use weighted facts?

Answer: Step 1 (fact decomposition): an LLM converts the free-form output into a list of atomic, verifiable claims (e.g., “Teddy bears were first created in the 1920s” as a separate claim). Step 2 (fact checking): each claim is verified against a trusted source (Wikipedia RAG, web search) and labeled correct (1) or incorrect (0). Aggregation: $\sum_i \alpha_i c_i$ where α_i are importance weights summing to 1. Weights are necessary because not all facts are equally important: incorrectly stating a founding year is less severe than incorrectly attributing the invention. Uniform weights are the simple default; domain knowledge informs non-uniform weights. This gives a nuanced score rather than binary pass/fail for the whole text.

L2 — What is pass-hat-k and why is it relevant for agentic benchmarks?

Answer: pass-hat-k (written $\widehat{\text{pass@}k}$) is the probability that *all* k attempts succeed, as opposed to pass@k which is the probability that *at least one* succeeds. For user-facing agentic tasks (e.g., changing a flight reservation, canceling an order), users interact with the system once. If the agent succeeds 70% of the time, 30% of real users get a wrong outcome.

This is unacceptable. We need *reliability*: every attempt should succeed. pass-hat-k directly measures this reliability. For example, if $\text{pass}@1 = 0.7$, then $\text{pass-hat-5} = 0.7^5 \approx 0.17$ — the system would give fully correct end-to-end behavior only 17% of the time when tested 5 times.

L3 — Compare BLEU, METEOR, and LLM-as-a-Judge. When would you use each?

Answer: BLEU: precision-based n-gram overlap + brevity penalty. Fast, reproducible, no labels needed at inference time. Use only for machine translation where references are available and acceptable outputs are relatively constrained. METEOR: similar to BLEU but adds recall, synonym matching, and ordering penalty. Slightly better correlation with human ratings. Same use case as BLEU but marginally better for tasks requiring linguistic flexibility. LLM-as-a-Judge: uses a language model to evaluate along any criterion. No reference needed; interpretable; flexible to any task or criterion. Best for: modern LLMs generating open-ended text (chat, code, creative writing, QA) where references are unavailable or output diversity makes n-gram metrics meaningless. Tradeoffs: LaaJ requires a separate LLM call (cost), has its own biases (position, verbosity, self-enhancement), and must be calibrated against human ratings to ensure it tracks ground truth.

L1 — What is Goodhart’s Law and how does it apply to LLM benchmarks?

Answer: Goodhart’s Law: “When a measure becomes a target, it ceases to be a good measure.” In LLMs: once a benchmark score (e.g., MMLU, GSM8K) becomes the optimization target during training, models learn shortcuts specific to the benchmark format without improving the underlying capability. Example: training on math-competition-style problems can inflate AIME scores without improving general mathematical reasoning. Remedy: use multiple benchmarks across diverse formats; complement with human evaluation (Chatbot Arena); regularly introduce new benchmarks with no contamination risk (e.g., AIME 2025); and evaluate on your own real-world task distribution rather than assuming benchmark performance generalizes.

Self-Assessment Test

Scoring: 16–20 = solid mastery; 12–15 = revisit weak spots; <12 = re-study.

Multiple Choice (1 point each)

Q1. Cohen’s kappa equals 0 when:

- (a) Raters never agree (b) Agreement is exactly at chance level (c) Raters always agree (d) Only one rater is evaluated

Q2. Which rule-based metric is precision-focused and includes a brevity penalty?

- (a) ROUGE (b) METEOR (c) BLEU (d) Cohen’s kappa

Q3. In LLM-as-a-Judge, why is rationale output before score?

- (a) To reduce latency (b) To enable chain-of-thought reasoning before committing to a score, improving quality (c) To make the output longer (d) Because providers require it in structured output format

Q4. MMLU evaluates LLMs on:

- (a) Multi-turn reasoning chains (b) 57 diverse multiple-choice tasks spanning many knowledge domains (c) Code generation and test passing (d) Safety and harmful behavior

Q5. A tool returns **None** instead of a result. What is the recommended practice?

- (a) Raise a Python exception (b) Return an empty JSON {} with a meaningful message (c) Retry the tool call 3 times (d) Let the LLM infer the result from context

Q6. The HarmBench attack success rate (ASR) measures:

- (a) How often the model answers correctly (b) How often the model produces harmful behavior when prompted adversarially (c) How many tools the model can call (d) The model’s factuality score

Q7. pass-hat-k is appropriate (over pass@k) when:

- (a) You can afford multiple tries and only need one success (b) You require consistent, reliable success across all attempts (c) The task is math-only (d) You are using a reasoning model

Q8. Verbosity bias in LLM-as-a-Judge refers to:

- (a) The judge preferring responses with more technical terms (b) The judge favoring longer responses regardless of quality (c) The generator producing overly long outputs (d) Response latency increasing with output length

Short Answer (1.5 points each)

Q9. State the formula for Cohen’s kappa and explain what “agreement by chance” means in a binary (yes/no) rating setup with two raters.

Q10. Describe the two-step factuality evaluation pipeline. What technique from Lecture 7 is used in Step 2?

Q11. An LLM agent calls `find_bear()` when the available tool is `find_teddy_bear()`. What failure mode is this? List two possible causes and their remedies.

Q12. Why does SWE-bench use test pass/fail as its evaluation criterion instead of LLM-as-a-Judge? What advantage does this give?

Q13. Explain data contamination in the context of benchmarks and describe two precautions against it.

Q14. What is the Pareto frontier in the context of model selection? Give two concrete trade-off axes you would plot.

Scenario (2 points)

Q15. You are the ML lead building an LLM-based customer support system for an e-commerce company. Your system handles order status inquiries, return requests, and product questions. After 2 weeks in production, you notice: (a) customers complain the system sometimes gives wrong order dates, (b) the system occasionally says “I cannot help with that” for legitimate requests, and (c) LaaJ scores are high (85% pass rate) but user satisfaction is only 70%. Design a complete evaluation and debugging strategy covering: (1) what evaluation dimensions you’d measure and with what tools, (2) how you’d debug complaints (a) and (b), (3) why there might be a gap between LaaJ scores and user satisfaction, and (4) one benchmark you’d use for ongoing monitoring.

Answer Key

Q1. (b) Kappa = 0 when observed agreement exactly equals chance agreement: no better than random.

Q2. (c) BLEU: precision-based (counts matching n-grams in the prediction) with a brevity penalty. ROUGE is recall-based; METEOR uses F-score + ordering penalty.

Q3. (b) Writing the rationale first forces chain-of-thought reasoning before committing to a score, empirically improving judgment quality — the same principle as reasoning models.

Q4. (b) 57 diverse multiple-choice tasks across many knowledge domains, each with 4 answer choices. Hardcoded match evaluation.

Q5. (b) Return {} (empty JSON) or a structured “not found” response. None tells the LLM nothing; an empty structured response tells it “I ran but found nothing.”

Q6. (b) ASR = fraction of adversarial prompts that successfully elicit harmful behavior. Evaluated by a classifier, not hardcoded match.

Q7. (b) pass-hat-k is appropriate when every attempt must succeed — e.g., real users each get one interaction and all should get correct outcomes.

Q8. (b) Verbosity bias: the judge prefers longer responses regardless of quality, conflating length with helpfulness.

Q9. $\kappa = (P_{\text{obs}} - P_{\text{chance}}) / (1 - P_{\text{chance}})$. In a binary setup with rater A saying yes with probability p_A and rater B with p_B : $P_{\text{chance}} = p_A p_B + (1 - p_A)(1 - p_B)$ — the probability they agree if each rates independently at random. Kappa subtracts this baseline and normalizes, so $\kappa = 0$ means random agreement, $\kappa = 1$ means perfect.

Q10. Step 1 (fact decomposition): an LLM converts the output text into a list of atomic, binary claims. Step 2 (fact checking): each claim is verified against trusted sources using RAG (from Lecture 7) — retrieve relevant documents and determine if the claim is supported. Aggregate with $\sum_i \alpha_i c_i$.

Q11. Tool hallucination failure mode. Causes: (a) model too weak / doesn’t understand the available tools; (b) API naming is non-intuitive (“find_teddy_bear” vs. what the model expected). Remedies: (a) upgrade the LLM or iterate on top-level instructions clarifying what functions are available; (b) rename the API to a more intuitive name, revamp the docstring.

Q12. Test pass/fail is deterministic and requires no subjective judgment — the code either works or it doesn’t. This eliminates LaaJ bias, needs no judge prompt engineering, and is perfectly reproducible. It also directly measures what matters: functional correctness, not stylistic quality.

Q13. Data contamination: benchmark examples appear in training data, so the model memorizes answers rather than demonstrating genuine capability. Precautions: (1) *Hash identifiers* — assign a unique hash to each benchmark example and check if it appears in the training corpus; (2) *Evaluate on new test versions* that post-date training (e.g., AIME 2025 for models trained on pre-2025 data).

Q14. The Pareto frontier is the set of models that are non-dominated: no model in the set can be improved on one objective without getting worse on another. Two trade-off axes: (1) quality (e.g., MMLU accuracy or LaaJ pass rate) vs. cost (price per million tokens) — the frontier shows the cheapest model for each quality level; (2) quality vs. safety (ASR) — the frontier shows the most capable model at each safety constraint.

Q15. (1) **Evaluation dimensions:** (a) Factuality (order dates) — use the two-step factuality pipeline: fact decompose + RAG against the order database; (b) Tool use correctness — did the

agent call the right tool with the right arguments?; (c) Helpfulness — binary LaaJ pass/fail; (d) Safety — LaaJ for harmful refusals. (2) **Debugging (a) wrong dates:** Instrument the agent to log the full tool call chain. Check if the order lookup tool is returning correct data (Step 2 failure: wrong tool response) or if the LLM is ignoring the tool’s result (Step 3: response generation error). Run factuality pipeline on a sample of 50 responses. (b) Punting: check if the tool router is selecting the right tool (recall failure) or if the LLM doesn’t know how to use it (SFT/prompt gap). Add test cases covering these query types. (3) **LaaJ vs. satisfaction gap:** LaaJ may have verbosity bias (scoring long responses highly even when customers find them confusing), or the judge criteria don’t match customer needs (customers want speed and actionability, not just “technically correct”). Recalibrate the judge by collecting 100 user satisfaction ratings and running correlation analysis. (4) **Ongoing benchmark:** Custom eval set of 200 real customer queries with ground truth labels (correct answers from the order database + known-good responses). Run weekly to detect regressions.

End of Study Guide — Lecture 8 (Final Lecture)

Source: Stanford CME 295, Fall 2025. Afshine & Shervine Amidi.

Good luck on the final! Lectures 5–8 are the covered material.